

Variable Types

Difference Between Class Variable, Instance Variable, and Local Variable

In Java, variables are categorized based on their scope, lifetime, and association with objects or classes. The three types of variables are **class variables**, **instance variables**, and **local variables**.

1. Class Variable

- **Definition:** A class variable is a variable that is declared with the `static` keyword inside a class but outside of any method, constructor, or block. It is shared by all instances of the class.
- **Scope:** The scope of a class variable is the entire class. It is accessible by all methods and can be accessed using the class name or an instance of the class.
- **Lifetime:** Class variables are created when the class is loaded into memory and destroyed when the class is unloaded.
- **Initialization:** Class variables are initialized once and shared among all instances of the class.
- **Access:** Can be accessed using the class name or through an instance of the class (though using the class name is recommended).
- **Example:**

```
class Car {
    static int totalCars = 0; // Class variable

    Car() {
        totalCars++; // Increment totalCars whenever a new object is
created
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(Car.totalCars); // Access class variable
        Car car1 = new Car();
        Car car2 = new Car();
        System.out.println(Car.totalCars); // Output: 2 (shared between
objects)
```

```
}  
}
```

2. Instance Variable

- **Definition:** An instance variable is a variable declared in a class but outside any method, constructor, or block. Each instance (object) of the class has its own copy of the instance variable.
- **Scope:** The scope of an instance variable is the entire class, but its value is specific to each object created from the class.
- **Lifetime:** Instance variables are created when an object is instantiated and destroyed when the object is garbage collected.
- **Initialization:** Instance variables are initialized when an object is created. They can have default values (e.g., `0` for numbers, `null` for objects).
- **Access:** Instance variables are accessed using an object reference.
- **Example:**

```
class Car {  
    String color; // Instance variable  
    int speed;    // Instance variable  
  
    Car(String color, int speed) {  
        this.color = color;  
        this.speed = speed;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car car1 = new Car("Red", 60);  
        Car car2 = new Car("Blue", 80);  
  
        // Access instance variables through the object  
        System.out.println(car1.color + " " + car1.speed); // Red 60  
        System.out.println(car2.color + " " + car2.speed); // Blue 80  
    }  
}
```

3. Local Variable

- **Definition:** A local variable is a variable declared inside a method, constructor, or block. It is only accessible within the method or block where it is declared.
- **Scope:** The scope of a local variable is limited to the method, constructor, or block in which it is declared.
- **Lifetime:** Local variables are created when the method or block is called and destroyed when the method or block finishes execution. They do not retain their values between method calls.
- **Initialization:** Local variables must be initialized before they are used. If they are not initialized, the compiler will throw an error.
- **Access:** Local variables are accessed only within the method or block where they are declared.
- **Example:**

```
class Car {  
    void displaySpeed() {  
        int speed = 100; // Local variable  
        System.out.println("Speed: " + speed);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car();  
        car.displaySpeed(); // Access the local variable inside the method  
    }  
}
```

Comparison Table

Feature	Class Variable	Instance Variable	Local Variable
Declaration	Declared with the <code>static</code> keyword in a class	Declared inside a class, outside methods, constructors	Declared inside methods, constructors, or blocks
Access	Accessed via the class name or instance	Accessed through an object instance	Accessed only within the method/block where declared
Scope	Class-wide (shared by all instances)	Specific to each object of the class	Limited to the method/block where declared
Lifetime	Lifetime of the class (created	Lifetime of the object (created when the	Lifetime of the method/block (created

Feature	Class Variable	Instance Variable	Local Variable
	when class is loaded, destroyed when class is unloaded)	object is created, destroyed when object is garbage collected)	when the method/block is called, destroyed when the method/block finishes)
Initialization	Initialized once, shared by all objects	Initialized separately for each object	Must be initialized before use
Default Value	Default value depends on type (e.g., <code>0</code> for int, <code>null</code> for reference types)	Default value depends on type (e.g., <code>0</code> for int, <code>null</code> for reference types)	No default value; must be manually initialized
Usage	Shared across all instances of the class	Holds instance-specific data	Temporarily holds data within a method/block
